Los Sabrossos

Contents

1. Math

1.1. Sieve of Eratosthenes

```
const int N = 1e7;
int sieve[N];
void make(){
    for(int i = 0; i < N; i++) sieve[i] = 1;
    sieve[0] = sieve[1] = 0;
    for (int i = 2; i < N; i++)
    {
        if(sieve[i]){
            for (int j = i*i; j < N; j+=i)
            {
                sieve[j] = 0;
            }
        }
    }
}
```

1.2. Euler Phi

```
ll phi(ll n) {
    ll ans = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            ans -= ans / p;
            while (n % p == 0){
                n /= p;
            }
        }
    }
    if (n > 1){
        ans -= ans / n;
    }
    return ans;
}
```

1.3. Binary Exponentiation

```
long long pote(long long a, long long b, long long mod){
    long long ans = 1ll;
    while(b){
        if(b&1) ans = (ans * a)%mod;
        a = (a * a)%mod;
        b>>=1;
    }
    return ans;
}
// a^-1 = a^mod-2 = pote(a,mod-2);
```

1.4. Chinese Rest Theorem

```
//si los modulos son muy grandes 1e18, cambiar por int128 las operaciones
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) {
        x = 1;
        y = 0;
        return a;
    }

    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return g;
}

// x      a (mod m)
// x      b (mod n)
//si retorna {0, -1} -> no solution
pair<ll,ll> CRT(ll a, ll m, ll b, ll n) {
    ll x, y;
```

```
    ll g = extgcd(m, n, x, y);

    if ((b - a) % g != 0)
        return {0, -1};

    ll n2 = n / g;

    ll t = ((b - a) / g * x) % n2;
    if (t < 0) t += n2;

    ll mod = m * n2;
    ll r = (a + m * t) % mod;
    if (r < 0) r += mod;

    return {r, mod};
}

// x     a[i] (mod m[i]) para todo i
//vector de valores y vector de respectivos modulos
pair<ll,ll> CRT(vector<ll> a, vector<ll> m) {
    ll r = a[0];
    ll mod = m[0];

    for (int i = 1; i < a.size(); i++) {
        pair<ll,ll> res = CRT(r, mod, a[i], m[i]);

        if (res.second == -1)
            return {0, -1};

        r = res.first;
        mod = res.second;
    }

    return {r, mod};
}

void solve(){
    vector<ll>a(2);
    vector<ll>m(2);
    repl(i,0,2){
        cin >> a[i] >> m[i];
    }
    pair<ll,ll> ans = CRT(a, m);
    if(ans.second == -1){
        cout << "-1\n";
    }
    else{
        cout << ans.first << " " << ans.second << "\n";
    }
}
```

1.5. Extended Euclides

```
//Encuentra
//valores para x e y
//A*x + B*y = gcd(A, B)
ll extgcd(ll a, ll b, ll &x, ll &y){
    if(b == 0){
        x = 1;
        y = 0;
        return a;
    }

    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

void solve(){
    ll A, B, C;
    cin >> A >> B >> C;
    ll x, y;
    ll g = extgcd(A, B, x, y);
    if(C % g != 0){
        cout << "-1\n";
        return;
    }
    ll k = -C / g;
    x *= k;
    y *= k;
    cout << x << " " << y << '\n';
}
```

1.6. Fibonacci Log(n)

```
long long int fibo(int n) {
    long long int a, b, x, y, tmp;
    a = x = tmp = 0;
    b = y = 1;
    while (n != 0)
        if (n % 2 == 0) {
            tmp = a * a + b * b;
            b = b * a + a * b + b * b;
            a = tmp;
            n = n / 2;
        } else {
            tmp = a * x + b * y;
            y = b * x + a * y + b * y;
            x = tmp;
            n = n - 1;
        }
    // x = fibo(n-1)
    return y; // y = fibo(n)
}
```

1.7. Least Common Multiple

```
ll lcm(ll a, ll b){
    return (b/__gcd(a,b)) * a;
}
```

1.8. Modular Inverse

```
const long long N = 1e6 + 10;
const int MOD = 1e9+7;
long long fact[N];
long long invfact[N];
void init(){
    fact[0] = 1;
    for(long long i = 1; i < N; i++)
        fact[i] = fact[i-1] * i % MOD;

    invfact[N-1] = pote(fact[N-1], MOD-2);

        for(long long i = N-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % MOD;
}

long long nCk(long long n, long long k){
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

1.9. Pollard Rho Rabin Miller

```
map<ll, ll> fact;

ll pote(ll a, ll b, ll mod) {
    ll ans = 1;
    while (b) {
        if (b & 1) ans = (__int128)ans * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return ans;
}

ll mcd(ll a, ll b) {
    a = abs(a);
    b = abs(b);
    while (a != 0) {
        ll r = b % a;
        b = a;
        a = r;
    }
    return b;
```

```
}

//Rabin Miller
bool primo(ll n) {
    if (n < 2) return false;
    if (n == 2) return true;
    ll D = n - 1, S = 0;
    while (D % 2 == 0) {
        D /= 2;
        S++;
    }
    static const ll STEPS = 16ll;
    for(ll pasos = 0; pasos < STEPS; pasos++){
        const ll A = 1 + rand() % (n - 1);
        ll M = pote(A, D, n);
        if (M == 1 || M == (n - 1)) goto next;
        for(ll k = 0; k < S-1; k++){
            M = (__int128)M * M % n;
            if (M == (n - 1)) goto next;
        }
        return false;
    next:;
    }
    return true;
}

//Rho de pollard
ll factor(ll n) {
    static ll A, B;
    A = 1 + rand() % (n - 1);
    B = 1 + rand() % (n - 1);
    #define fun(x) (((__int128)(x) * ((x) + B)) % n + A)

    ll x = 2, y = 2, d = 1;
    while (d == 1 || d == -1) {
        x = fun(x);
        y = fun(fun(y));
        d = mcd(x - y, n);
    }
    return abs(d);
}

void factorize(ll n){
    assert(n > 0);
    while (n > 1 && !primo(n)) {
        ll fx;
        do {
            fx = factor(n);
        } while (fx == n);

        n /= fx;
        factorize(fx);

        for (auto &it : fact) {
            while (n % it.first == 0) {
                n /= it.first;
                it.second++;
            }
        }
    }
    if (n > 1) fact[n]++;
}
```

2. Geometry

2.1. Convex Hull

```
// Devuelve sentido horario
vector<point> hull(vector<point> p)
{
    int n = p.size();
        vector<point> h;
        sort(p.begin(),p.end());
        for(int i = 0; i < n; i++){
                while(h.size() >= 2 and p[i].left(h[h.size () - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.push_back(p[i]);
        }
        h.pop_back();
        int k = h.size();
```

```
        for(int i = n-1; i >= 0; i--)
        {
                while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.pb(p[i]);
        }
        h.pop_back();
        return h;
}
```

2.2. Point

```
struct point
{
        int x, y;
        point() {}
        point(int x, int y): x(x), y(y) {}
        point operator -(point p) {return point(x - p.x, y - p.y);}
        point operator +(point p) {return point(x + p.x, y + p.y);}
        int sq() const{return x * x + y * y;}
        double abs() const {return sqrt(sq());}
        int operator ^(point p) {return x * p.y - y * p.x;}
        int operator *(point p) {return x * p.x + y * p.y;}
        point operator *(int a) {return point(x * a, y * a);}
        bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
        int left(point a, point b) {
        return ((b - a) ^ (*this - a)); // 0, -, +
    }
};
ostream& operator<<(ostream& os, const point& p) {
    return os << "(" << p.x << "," << p.y << ")\n";
}


void polarSort(vector<point>& v) {
  sort(v.begin(), v.end(), [] (point a, point b) {
    const point origin{0, 0};
    bool ba = a < origin, bb = b < origin;
    if (ba != bb) { return ba < bb; }
    return (a^b) > 0;
  });
}
```

2.3. Closest Pair of Points Lineal

```
struct pt {
    ll x, y;
    bool operator == (const pt & o) const {
        return x == o.x and y == o.y;
    }
};

struct CustomHashPoint {
    size_t operator()(const pt & p) const {
        static
        const uint64_t C =
            chrono::steady_clock::now().time_since_epoch().count();
        uint64_t x = p.x;
        uint64_t y = p.y;
        x ^= x >> 30;
        x *= 0xbf58476d1ce4e5b9;
        x ^= x >> 27;
        x *= 0x94d049bb133111eb;
        x ^= x >> 31;
        y ^= y >> 30;
        y *= 0xbf58476d1ce4e5b9;
        y ^= y >> 27;
        y *= 0x94d049bb133111eb;
        y ^= y >> 31;
        return C ^ (x + (y << 1));
    }
};

ll dist2(pt a, pt b) {
    ll x = a.x - b.x;
    ll y = a.y - b.y;
    return x * x + y * y;
}

pair < int, int > closest_pair(vector < pt > p) {
```

```
    int n = p.size();
    unordered_map < pt, int, CustomHashPoint > mp;
    // puntos repetidos
    for (int i = 0; i < n; i++) {
        if (mp.count(p[i]))
            return {
                mp[p[i]],
                i
            };
        mp[p[i]] = i;
    }
    mt19937 rng(
        chrono::steady_clock::now().time_since_epoch().count()
    );
    auto rnd = [ & ]() {
        return uniform_int_distribution < int > (0, n - 1)(rng);
    };
    ll d2 = dist2(p[0], p[1]);
    pair < int, int > ans = {
        0,
        1
    };
    auto upd = [ & ](int i, int j) {
        ll nd = dist2(p[i], p[j]);
        if (nd < d2) {
            d2 = nd;
            ans = {
                i,
                j
            };
        }
    };
    // conseguir una distancia inicial razonable
    for (int it = 0; it < 20; it++) {
        int i = rnd();
        int j = rnd();
        while (i == j) j = rnd();
        upd(i, j);
    }
    ll d = sqrtl((long double) d2) + 1;
    unordered_map < pt, vector < int > , CustomHashPoint > grid;
    grid.reserve(2 * n);
    for (int i = 0; i < n; i++) {
        grid[{
            p[i].x / d,
            p[i].y / d
        }].push_back(i);
    }
    for (auto & [c, v]: grid) {
        // misma celda
        for (int i = 0; i < (int) v.size(); i++) {
            for (int j = i + 1; j < (int) v.size(); j++) {
                upd(v[i], v[j]);
            }
        }
        // celdas vecinas
        for (int dx = -1; dx <= 1; dx++) {
            for (int dy = -1; dy <= 1; dy++) {
                if (dx == 0 and dy == 0) continue;
                pt to = {
                    c.x + dx,
                    c.y + dy
                };
                auto it = grid.find(to);
                if (it == grid.end()) continue;
                for (int i: v) {
                    for (int j: it -> second) {
                        upd(i, j);
                    }
                }
            }
        }
    }
    return ans;
}
```

2.4. Closest Pair of Points Nlogn

```
struct pt {
    ll x, y;
    int id;
};

ll dist2(pt a, pt b) {
```

```
    ll x = a.x - b.x;
    ll y = a.y - b.y;
    return x * x + y * y;
}

pair < int, int > closest_pair(vector < pt > p) {
    int n = p.size();
    sort(p.begin(), p.end(), [](pt a, pt b) {
        if (a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    });
    set < pair < ll, int >> s; // {y,id}
    ll d2 = dist2(p[0], p[1]);
    pair < int, int > ans = {
        p[0].id,
        p[1].id
    };
    int l = 0;
    for (int i = 0; i < n; i++) {
        ll d = sqrtl((long double) d2) + 1;
        // sacar puntos que ya est n demasiado lejos en x
        while (l < i and p[i].x - p[l].x >= d) {
            s.erase({
                p[l].y,
                p[l].id
            });
            l++;
        }
        // buscar puntos con y cerca
        auto it = s.lower_bound({
            p[i].y - d,
            -1
        });
        while (it != s.end() and it -> first <= p[i].y + d) {
            int j = it -> second;
            ll nd = dist2(p[i], p[j]);
            if (nd < d2) {
                d2 = nd;
                ans = {
                    p[i].id,
                    p[j].id
                };
                d = sqrtl((long double) d2) + 1;
            }
            it++;
        }
        s.insert({
            p[i].y,
            p[i].id
        });
    }
    return ans;
}
```

3. Strings

3.1. KMP

```
vector<int> kmp(string s){
    int n = s.size();
    vector<int> pi(n);
    for(int i = 1; i < n; i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j]){
            j = pi[j-1];
        }
        if(s[i] == s[j]){
            j++;
        }
        pi[i] = j;
    }
    return pi;
}
//Pattern Matching
vector<int> ocurrencia(string texto, string patron) {
    vector<int> P = kmp(patron);
    int k = 0;
    vector<int> M;
    repl(i,0,texto.size()) {
        while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
        if (patron[k] == texto[i]) k++;
```

```
        if (k == patron.size()) {
            M.pb(i - k + 1);
            k = P[k - 1];
        }
    }
    return M;
}

//Automata
void genAut(string &s) {
    int n = s.size();

    vector<vector<int>> aut(n+1, vector<int>(26));
    vector<int> pi = kmp(s);

    for(int i = 0 ; i < n; i++){
        aut[i][s[i]-'a'] = i+1;
    }

    for(int i = 0; i < n+1; i++){
        for(int c = 0; c < 26; c++){
            if (aut[i][c]) continue;
            if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
            else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
            else aut[i][c] = 0;
        }
    }
}
```

3.2. Z Function

```
vector<int> z_function(string s){
    int n = s.size();
    vector<int> z(n,0);
    int l = 0;
    int r = 0;
    repl(i,1,n){
        if(i < r){
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n and s[z[i]] == s[i + z[i]]){
            z[i]++;
        }
        if(i + z[i] > r){
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

3.3. Trie

```
const int E = 26;

struct AC {
    int nodes;
    vector<array<int, E>>go;
    vector<bool>terminal;

    AC(){
        add_node();
        nodes = 1;
    }

    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - '0';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
```

```
    }

};
```

3.4. Suffix Array

```
vector<int> suffix_array(string s) {
    s += char(0);
    const int n = s.size();
    vector<int>a(n);
    iota(a.begin(), a.end(), 0);
    sort(a.begin(), a.end(), [&] (int i, int j){
        return s[i] < s[j];
    });
    vector<int> mapping(n);
    mapping[a[0]] = 0;
    for(int i = 1; i < n; i++){
        mapping[a[i]] = mapping[a[i-1]]+(s[a[i-1]] != s[a[i]]);
    }
    int len = 1;
    vector<int>sbs(n);
    vector<int>head(n);
    vector<int>new_mapping(n);
    while(len < n){
        for(int i = 0; i<n; i++) sbs[i] = (a[i] + n - len) % n;
        for(int i = n-1; i>=0; i--) head[mapping[a[i]]] = i;
        for(int i = 0; i < n; i++){
            int x = sbs[i];
            a[head[mapping[x]]++] = x;
        }
        new_mapping[a[0]] = 0;
        for(int i = 1; i < n; i++){
            if(mapping[a[i-1]] != mapping[a[i]]){
                new_mapping[a[i]] = new_mapping[a[i-1]]+1;
            }
            else{
                int pre = mapping[(a[i-1] + len)%n];
                int cur = mapping[(a[i]+len) % n];
                new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
            }
        }
        len <<= 1;
        swap(mapping, new_mapping);
    }
    return vector<int>(a.begin()+1, a.end());//ignorar a[0] que es el centinela
}

vector<int> lcp_array(vector<int>&a, string s){
    const int n = s.size();
    vector<int> rank(n);
    for(int i = 0; i < n; i++) rank[a[i]] = i;
    int k = 0;
    vector<int>lcp(n);
    for(int i = 0; i < n; i++){
        if(rank[i] == n-1){
            k = 0;
            continue;
        }
        int j = a[rank[i]+1];
        while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    lcp.pop_back();
    return lcp;
}

void solve(){
    //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
    //verificar si un string es substring de otro con BS en SA
    //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
    string s, t;
    cin >> s >> t;
    if(s.size() > t.size()){
        swap(s, t);
    }
    int n = s.size();
    int m = t.size();
    string st = s+'#'+t;
    vector<int>sa = suffix_array(st);
    vector<int>lcp =lcp_array(sa, st);
    // print(sa);
    // print(lcp);
    vector<int>tr(n+m);
```

```
    repl(i,0,n+m){
        if(sa[i] >=n and sa[i+1]>=n){
            tr[i] = 0;
            continue;
        }
        if(sa[i] < n and sa[i+1] < n){
            tr[i] = 0;
            continue;
        }
        int mn = min(sa[i], sa[i+1]);
        int tam1 = mn + lcp[i];
        // dbg(tam1);
        tr[i] = lcp[i];
        if(tam1 > n){
            dbg(tam1);
            tr[i] = lcp[i] - (tam1-n);
        }
    }
    // print(tr);
    int mx = *max_element(tr.begin(), tr.end());
    repl(i,0,n+m){
        // dbg(tr[i]);
        if(tr[i] == mx){
            string ans = st.substr(sa[i], mx);
            cout << ans << '\n';
            return;
        }
    }

}
//Sabrossus
```

3.5. Aho Corasick

```
//AhoCorasick para saber si un string s existe en un texto T
const int E = 26;
const int N = 1e6+10;
int cur;
int nodoTerminal[N];//se guarda el nodo terminal del string i
set<int>st;//se guarda que nodos son terminales que aparecen en el texto T

struct AC {
    int nodes;
    vector<int>suf, super;
    vector<array<int, E>>go;
    vector<bool>terminal;
    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
        super.emplace_back();
        suf.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
```

```
                else{
                    go[u][c] = u == 0 ? 0 :go[suf[u]][c];
                }
            }
        }
    }
};

void mark(int u, AC &ac){
    if(u == 0) return;
    mark(ac.super[u], ac);
    if(ac.terminal[u]){
        st.insert(u);
        ac.terminal[u] = false;
    }
    ac.super[u] = 0;
}

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t; //texto grande
    cin >> t;
    ac.build();
    int p = 0;
    for(auto x : t){
        int c = x - 'a';
        p = ac.go[p][c];
        mark(p, ac);
    }
    repl(i,0,n){
        if(st.count(nodoTerminal[i])){
            cout << "YES\n";
        }
        else{
            cout << "NO\n";
        }
    }
}

//-------------------------------------------------------------------------

//AhoCorasick para Frecuencias
const int E = 26;
const int N = 1e6+10;
int nodoTerminal[N];
int cur;
struct AC {
    int nodes;
    vector<int>suf, freq, by_level;
    vector<bool>terminal;
    vector<array<int,E>>go;

    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        suf.emplace_back();
        go.emplace_back();
        freq.emplace_back();
    }

    void insert(string &s){
        int pos = 0;
        for(char ch:s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] = true;
        nodoTerminal[cur] = pos;
    }

    void build(){
```

```
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            by_level.emplace_back(u);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 : go[suf[u]][c];
                }
            }
        }
    }
};

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t;
    cin >> t;
    ac.build();
    int p = 0;
    for(char ch:t){
        int c = ch-'a';
        p = ac.go[p][c];
        ac.freq[p]++;
    }
    for(int i = (int)ac.by_level.size()-1; i > 0; i--){
        int x = ac.by_level[i];
        ac.freq[ac.suf[x]] += ac.freq[x];
    }
    repl(i,0,n){
        int termi = nodoTerminal[i];
        cout << ac.freq[termi] << '\n';
    }
}
```

3.6. Booths Algorithm

```
//Algoritmo para menor rotacion lexicograf ca de un string en O(N)

int menorRotacion(string &s){
    int n = s.size();
    vector<int>f(2*n, -1);
    int k = 0; //indice mejor rotacion
    for(int j = 1; j < 2*n;j++){
        int i = f[j-k-1];
        //el operador %n simula la concatenacion de la cadena s+s
        while(i!=-1 and s[j%n]!=s[(k+i+1)%n]){
            if(s[j%n] < s[(k+i+1)%n]){
                k = j - i - 1;
            }
            i = f[i];
        }
        if(i == -1 and s[j%n] != s[(k+i+1)%n]){
            if(s[j % n] < s[(k+i+1%n)]){
                k = j;
            }
            f[j - k] = -1;
        }
        else{
            f[j - k] = i + 1;
        }
    }
    return k;
}
void solve(){
    string s;
    cin >> s;
    ll n = s.size();
    ll ans = menorRotacion(s);
    repl(i, ans, ans+n){
```

```
        cout << s[i%n];
    }
}
```

3.7. Hashing 2D

```
/*
    2D Hashing: buscar todas las apariciones de una matriz
    r x c dentro de una matriz n x m.
*/

const ll BASE = 911382323LL;

void solve() {
    int r, c;
    cin >> r >> c;

    vector<string> pattern(r);
    for (int i = 0; i < r; i++)
        cin >> pattern[i];

    // Potencias para el rolling vertical.
    vector<ll> power(r + 1, 1);
    for (int i = 1; i <= r; i++)
        power[i] = power[i - 1] * BASE;

    // Hash de la matriz patr n.
    ll patternHash = 0;

    for (int i = 0; i < r; i++) {
        Hash h(pattern[i]);
        ll rowHash = h.get(0, c);

        patternHash += rowHash * power[r - i - 1];
    }

    int n, m;
    cin >> n >> m;

    vector<string> grid(n);
    for (int i = 0; i < n; i++)
        cin >> grid[i];

    // rowHash[i][j] = hash de grid[i][j ... j+c-1].
    vector<vector<ll>> rowHash(
        n, vector<ll>(m - c + 1)
    );

    for (int i = 0; i < n; i++) {
        Hash h(grid[i]);

        for (int j = 0; j <= m - c; j++)
            rowHash[i][j] = h.get(j, j + c);
    }

    // matrixHash[i][j] = hash de la submatriz r x c desde (i,j).
    vector<vector<ll>> matrixHash(
        n - r + 1,
        vector<ll>(m - c + 1)
    );

    for (int j = 0; j <= m - c; j++) {
        ll cur = 0;

        // Primera ventana vertical.
        for (int i = 0; i < r; i++)
            cur += rowHash[i][j] * power[r - i - 1];

        matrixHash[0][j] = cur;

        // Rolling vertical.
        for (int i = 1; i <= n - r; i++) {
            cur -= rowHash[i - 1][j] * power[r - 1];
            cur = cur * BASE + rowHash[i + r - 1][j];

            matrixHash[i][j] = cur;
        }
    }

    // 2D difference array para marcar las apariciones.
    vector<vector<int>> diff(
        n + 1, vector<int>(m + 1, 0)
    );

```

```
    for (int i = 0; i <= n - r; i++) {
        for (int j = 0; j <= m - c; j++) {

            if (matrixHash[i][j] == patternHash) {
                diff[i][j]++;
                diff[i][j + c]--;
                diff[i + r][j]--;
                diff[i + r][j + c]++;
            }
        }
    }

    // Prefijos 2D.
    for (int i = 0; i < n; i++)
        for (int j = 1; j < m; j++)
            diff[i][j] += diff[i][j - 1];

    for (int j = 0; j < m; j++)
        for (int i = 1; i < n; i++)
            diff[i][j] += diff[i - 1][j];

    // Imprimir celdas pertenecientes a alguna aparici n.
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << (diff[i][j] > 0 ? grid[i][j] : '.');
        }
        cout << '\n';
    }
}
```

3.8. Hashing

```
struct Hash {
        long long P=1777771,MOD[2],PI[2];
        vector<long long> h[2],pi[2];

        // SOLO PARA ROLLING / JOIN:
        vector<long long> pw[2];

        Hash(string& s){
                MOD[0]=999727999;
                MOD[1]=1070777777;
                PI[0]=325255434;
                PI[1]=10018302;

                for(int k = 0; k < 2; k++) {
                        h[k].resize(s.size()+1);
                        pi[k].resize(s.size()+1);

                        // SOLO PARA ROLLING / JOIN:
                        pw[k].resize(s.size()+1);
                }

                for(int k = 0; k < 2; k++){
                        h[k][0] = 0;
                        pi[k][0] = 1;

                        // SOLO PARA ROLLING / JOIN:
                        pw[k][0] = 1;

                        long long p = 1;

            for(int i = 1; i < s.size()+1; i++){
                                h[k][i] = (h[k][i-1]+p*s[i-1])%MOD[k];
                                pi[k][i] = (1LL*pi[k][i-1]*PI[k])%MOD[k];

                                // SOLO PARA ROLLING / JOIN:
                                pw[k][i] = (1LL*pw[k][i-1]*P)%MOD[k];

                                p = (p*P)%MOD[k];
                        }
                }
        }

        long long get(int s, int e){
                long long h0 = (h[0][e]-h[0][s]+MOD[0])%MOD[0];
                h0 = (1LL*h0*pi[0][s])%MOD[0];

                long long h1 = (h[1][e]-h[1][s]+MOD[1])%MOD[1];
                h1 = (1LL*h1*pi[1][s])%MOD[1];

                return (h0<<32)|h1;
        }

```

```
        // SOLO PARA ROLLING / JOIN:
        long long join(long long a, long long b, int lenA){
                long long a0 = a>>32;
                long long a1 = a&0xffffffffLL;

                long long b0 = b>>32;
                long long b1 = b&0xffffffffLL;

                a0 = (a0 + b0*pw[0][lenA])%MOD[0];
                a1 = (a1 + b1*pw[1][lenA])%MOD[1];

                return (a0<<32)|a1;
        }
};
```

3.9. Manacher

```
const int N = 1e5 + 10;
int odd[N]; //odd[i] = max odd palindrome centered on i
int even[N]; //even[i] = max even palindrome centered on i
void manacher(string & s) {
    int l = 0, r = -1, n = s.size();
    for (int i = 0; i < n; i++) {
        int k = i > r ? 1 : min(odd[l + r - i], r - i);
        while (i + k < n and i - k >= 0 and s[i + k] == s[i - k]) k++;
        odd[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    l = 0;
    r = -1;
    for (int i = 0; i < n; i++) {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        k++;
        while (i + k <= n and i - k >= 0 and s[i + k - 1] == s[i - k]) k++;
        even[i] = --k;
        if (i + k - 1 > r) l = i - k, r = i + k - 1;
    }
}
```

4. Bit Manipulation

4.1. Binary Representation

```
string tobit(long long x) {
    string a(64, '0');
    for (int i = 63; i >= 0; --i) {
        if (x & 1) a[i] = '1';
        x >>= 1;
    }
    return a;
}
```

4.2. Bit Mask

```
for(int mask = 0; mask < (1 << n); mask++) { //(1 << n) = O(2^n)
    for(int i = 0; i < n; i++) { // O(n)
    //verificamos si el i-esimo bit esta encendido
        if(mask & (1 << i)) {
            cout << v[i] << " ";
        }
    }
}
```

4.3. Bits Prendidos En Un Numero X En Una Posicion Pos

```
//retorna el numero de bits prendidos
//en una posicion pos desde 0 hasta el numero num
//tambien se puede usar en rangos asi:
//prendidos(R, 1) - prendidos(L-1, 1);
ll prendidos(ll num, ll pos){
    if(num <= 0){
        return 0;
```

```
    }
    ll pote = (1ll << pos);
    ll blocks = (num + 1) / pote;
    ll ones = blocks / 2;
    ones *= pote;
    if(blocks & 1){
        ones += (num + 1) % pote;
    }
    return ones;
}
//Sabrosson
```

4.4. Turn Off a Bit at Position X

```
void solve(){
    //mapa de caracteres en windows para ~
    int a, x;
    cin >> a >> x; // a es el numero, x es el numero de bit
    //los bits se enumeran de dercha a izquierda iniciando de 0
    /*
    Ej: n = 8
    Binario:     1 0 0 0
    Posicion:    3 2 1 0
    */
    a = a & ~(1 << x); //apagamos el bit x de a
    cout << a << "\n";
}
```

4.5. Xor Basis

```
ll base[LOG];
ll tam = 0;
void add(ll msk){
        for(int i = LOG-1; i>= 0; i--){
                if(!(msk&(1LL<<i))) continue;
                if(!base[i]){
                        base[i]=msk;
                        tam++;
                        return;
                }
                msk^=base[i];
        }
}
```

5. Data Structures

5.1. Segment Tree

```
const int N = 1e5;

struct info {
    int num;
};

info ST[4 * N];

info ope(info u, info v) {
    return {u.num + v.num};
}

void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in] = {v[L]}; // Asignamos el valor desde el vector
    } else {
        int mid = (L + R) >> 1;
        build(v, 2 * in, L, mid);
        build(v, (2 * in) + 1, mid + 1, R);
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

void update(int pos, int val, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in].num = val;
    } else {
```

```
        int mid = (L + R) >> 1;
        if (pos <= mid) {
            update(pos, val, 2 * in, L, mid);
        } else {
            update(pos, val, (2 * in) + 1, mid + 1, R);
        }
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

info get(int l, int r, int in = 1, int L = 0, int R = n - 1) {
    if (r < L or R < l) return {0}; // Elemento neutro
    if (l <= L and R <= r) return ST[in];

    int mid = (L + R) >> 1;
    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, (2 * in) + 1, mid + 1, R);
    return ope(left, right);
}


// Llamadas simples y limpias:
// build(v);              // Construye con el vector
// update(2, 10);         // Cambia el valor en pos 2 a 10
// info ans = get(1, 3); // Obtiene la respuesta en el rango [1, 3]
// v = {1 2 3 4 5}
// get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

5.2. Segment Tree Lazy Propagation

```
const int N = 1e5;

struct info {
    long long num;
};

info ST[4 * N];
long long lazy[4 * N];

info ope(info u, info v) {
    return {u.num + v.num};
}

void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in] = {v[L]};
    } else {
        int mid = (L + R) >> 1;

        build(v, 2 * in, L, mid);
        build(v, 2 * in + 1, mid + 1, R);

        ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
    }
}

void push(int in, int L, int R) {
    if (lazy[in] == 0)
        return;

    ST[in].num += lazy[in] * (R - L + 1);

    if (L != R) {
        lazy[2 * in] += lazy[in];
        lazy[2 * in + 1] += lazy[in];
    }

    lazy[in] = 0;
}

void update(int l, int r, long long val,
            int in = 1, int L = 0, int R = n - 1) {

    push(in, L, R);

    if (r < L or R < l)
        return;

    if (l <= L and R <= r) {
        lazy[in] += val;
        push(in, L, R);
        return;
    }

```

```
    int mid = (L + R) >> 1;

    update(l, r, val, 2 * in, L, mid);
    update(l, r, val, 2 * in + 1, mid + 1, R);

    ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
}

info get(int l, int r,
         int in = 1, int L = 0, int R = n - 1) {

    push(in, L, R);

    if (r < L or R < l)
        return {0};

    if (l <= L and R <= r)
        return ST[in];

    int mid = (L + R) >> 1;

    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, 2 * in + 1, mid + 1, R);

    return ope(left, right);
}
```

5.3. Bit 1D

```
struct BIT{
    int n;
    vector<ll> bit;
    BIT(int _n){
        n = _n;
        bit.assign(n + 1, 0);
    }
    void add(int x, ll val){
        for (int i = x; i <= n; i += (i&(-i))){
            bit[i] += val;
        }

    }
    ll sum(int x){
        ll ans = 0ll;
         for (int i = x; i > 0; i -= (i&(-i))){
            ans += bit[i];
        }
        return ans;
    }
    ll query(int l, int r){
        return sum(r) - sum(l - 1);
    }
};
// Para actulizar en rango y consultas puntuales
// bit.add(l,val); bit.add(r+1,val);
// bit.query(1,pos)
```

5.4. Bit 2D

```
struct BIT{
    int n, m;
    vector<vector<ll>> bit;

    BIT(int _n){
        n = _n;
        m = _n;
        bit.assign(n+1, vector<ll>(n+1 ,0));
    }
    void add(int x, int y, ll val){
        for (int i = x; i <= n; i += (i&(-i))){
            for (int j = y; j <= n; j += (j&(-j))){
                bit[i][j] += val;
            }
        }
    }
    ll sum(int x, int y){
        ll ans = 0ll;
        for (int i = x; i > 0; i -= (i&(-i))){
            for (int j = y; j > 0; j -= (j&(-j))){
                ans += bit[i][j];
            }
```

```
        }
        return ans;
    }
    ll query(int x1, int y1, int x2, int y2){
        return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) + sum(x1 - 1, y1 - 1);
    }
};
```

5.5. Bit 2D Infinite Plane

```
struct BIT{
    ll n;
    vector<vector<ll>> xs;
    vector<vector<ll>> bit;
    BIT(ll nn){
        n = nn;
        xs.resize(n + 1);
        bit.resize(n + 1);
    }
    void prepare(ll x, ll y){
        for (ll i = x; i <= n; i += (i&(-i))){
            xs[i].push_back(y);
        }
    }
    void build(){
        for (ll i = 1; i <= n; i++){
            sort(xs[i].begin(), xs[i].end());
            xs[i].erase(unique(xs[i].begin(), xs[i].end()), xs[i].end());
            bit[i].assign(xs[i].size() + 1, 0);
        }
    }
    void add(ll x, ll y, ll val){
        for (ll i = x; i <= n; i += (i&(-i))){
            ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin() + 1;
            for (ll j = p; j < bit[i].size(); j += (j&(-j))){
                bit[i][j] += val;
            }
        }
    }
    ll sum(ll x, ll y){
        ll ans = 0ll;
        for (ll i = x; i > 0; i -= (i&(-i))){
            ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin();
            for (ll j = p; j > 0; j -= (j&(-j))){
                ans += bit[i][j];
            }
        }
        return ans;
    }
    ll query(ll x1, ll y1, ll x2, ll y2){
        return sum(x2-1,y2) - sum(x2-1, y1) - sum(x1 - 1, y2) + sum(x1 - 1, y1);
    }

};
void solve(int tc){
    int n,q; cin >> n >> q;
    set<ll> X;
    set<ll> Y;
    vector<vector<ll>> points;
    vector<vector<ll>> query;
    repl(i,0,n){
        ll x,y,w; cin >> x >> y >> w;
        X.insert(x);
        Y.insert(y);
        points.push_back({x,y,w});
    }
    repl(i,0,q){
        int t; cin >> t;
        if(t == 0){
            ll x,y,w; cin >> x >> y >> w;
            X.insert(x);
            Y.insert(y);
            query.push_back({0,x,y,w});
        }
        else{
            ll x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            X.insert(x1);
            X.insert(x2);
            Y.insert(y1);
            Y.insert(y2);
            query.push_back({1,x1,y1,x2,y2});
        }
    }
```

```
    map<ll,ll> equis, ye;
    int i = 0;
    for(ll x : X){
        i++;
        equis[x] = i;
    }
    i = 0;
    for(ll x : Y){
        i++;
        ye[x] = i;
    }
    BIT bit((ll)X.size());
    for(auto &x : points){
        x[0] = equis[x[0]];
        x[1] = ye[x[1]];
        bit.prepare(x[0], x[1]);
    }
    for(auto &x : query){
        if(x[0] == 0){
            x[1] = equis[x[1]];
            x[2] = ye[x[2]];
            bit.prepare(x[1], x[2]);
        }
        else{
            x[1] = equis[x[1]];
            x[2] = ye[x[2]];
            x[3] = equis[x[3]];
            x[4] = ye[x[4]];
        }
    }
    bit.build();
    for(auto &x : points){
        bit.add(x[0], x[1], x[2]);
    }
    for(auto &x : query){
        if(x[0] == 0){
            bit.add(x[1], x[2], x[3]);
        }
        else{
            cout << bit.query(x[1],x[2],x[3],x[4]) << "\n";
        }
    }
}
```

5.6. Bit 3D

```
struct BIT {
    int n;
    int m;
    int p;
    vector < vector < vector < ll >>> bit;
    BIT(int _n) {
        n = _n + 10;
        m = _n + 10;
        p = _n + 10;
        bit.assign(n, vector < vector < ll >> (n, vector < ll > (n, 0)));
    }
    void add(int x, int y, int z, int val) {
        for (int i = x; i <= n; i += (i & (-i))) {
            for (int j = y; j <= n; j += (j & (-j))) {
                for (int k = z; k <= n; k += (k & (-k))) {
                    bit[i][j][k] += val;
                }
            }
        }
    }
    ll sum(int x, int y, int z) {
        ll ans = 0 ll;
        for (int i = x; i > 0; i -= (i & (-i))) {
            for (int j = y; j > 0; j -= (j & (-j))) {
                for (int k = z; k > 0; k -= (k & (-k))) {
                    ans += bit[i][j][k];
                }
            }
        }
        return ans;
    }
    ll query(int x1, int y1, int z1, int x2, int y2, int z2) {
        return sum(x2, y2, z2) - sum(x1 - 1, y2, z2) - sum(x2, y2, z1 - 1) - sum(x2, y1 - 1, z2
            ) + sum(x1 - 1, y1 - 1, z2) + sum(x2, y1 - 1, z1 - 1) + sum(x1 - 1, y2, z1 - 1) -
            sum(x1 - 1, y1 - 1, z1 - 1);
    }
};
```

5.7. DSU Dynamic Representative

```
int n, q;
cin >> n >> q;
DSU dsu(n + q + 10);
vector<int> id(n + 1),box(n + q + 10);
int nodes = n;
iota(id.begin(), id.end(), 0);
iota(box.begin(), box.end(), 0);
for(int i = 0 ;  i < q ; i ++){
    int a, b;
    cin >> a >> b;
    int ori = id[a];
    int ori2 = id[b];
    dsu.parent[ori] = ori2;
    nodes++;
    id[a] = nodes;
    box[nodes] = a;
}
for(int i = 1; i  <= n  ; i ++){
    cout << box[dsu.find(i)] << ' ';
}
```

5.8. Disjoint Set Union

```
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        //si tienes index 0 inicializar desde 0
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y] += sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x] += sizes[y];
        }

    }
};
//Sabrossus
```

5.9. Dynamic Connectivity Offline

```
struct Change {
    int a, b, sz_a;
};

struct DSU {
    vector<int> parent, sz;
    vector<Change> history;
    int components;

    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
```

```
        components = n;
    }

    int find(int x) {
        while (x != parent[x])
            x = parent[x];
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            history.push_back({-1, -1, -1});
            return false;
        }
        if (sz[a] < sz[b])
            swap(a, b);
        history.push_back({a, b, sz[a]});
        parent[b] = a;
        sz[a] += sz[b];
        components--;
        return true;
    }

    int snapshot() {
        return history.size();
    }

    void rollback(int snap) {
        while (history.size() > snap) {
            auto [a, b, old_sz_a] = history.back();
            history.pop_back();
            if (a == -1)
                continue;
            parent[b] = b;
            sz[a] = old_sz_a;
            components++;
        }
    }
};

struct Edge {
    int u, v;
};

struct SegmentTree {
    int n;
    vector<vector<Edge>> tree;
    SegmentTree(int n) {
        this->n = n;
        tree.resize(4 * n + 5);
    }
    void add(int node, int l, int r, int ql, int qr, Edge e) {
        if (qr < l or r < ql)
            return;
        if (ql <= l and r <= qr) {
            tree[node].push_back(e);
            return;
        }
        int mid = (l + r) / 2;
        add(node * 2, l, mid, ql, qr, e);
        add(node * 2 + 1, mid + 1, r, ql, qr, e);
    }
    void add(int l, int r, Edge e) {
        if (l > r)
            return;
        add(1, 1, n, l, r, e);
    }
};

void dfs(int node, int l, int r, SegmentTree &seg, DSU &dsu, vector<int> &answer){
    int snap = dsu.snapshot();
    for (auto [u, v] : seg.tree[node])
        dsu.unite(u, v);
    if (l == r) {
        answer[l] = dsu.components;
    } else {
        int mid = (l + r) / 2;
        dfs(node * 2, l, mid, seg, dsu, answer);
        dfs(node * 2 + 1, mid + 1, r, seg, dsu, answer);
    }
    dsu.rollback(snap);
}

void solve() {
```

```
    int n, q;
    cin >> n >> q;
    if (q == 0)
        return;
    DSU dsu(n);
    SegmentTree seg(q);
    map<pair<int,int>, int> start;
    vector<int> answer(q + 1);
    vector<bool> query(q + 1);
    for (int t = 1; t <= q; t++) {
        char type;
        cin >> type;
        if (type == '?') {
            query[t] = true;
        } else {
            int u, v;
            cin >> u >> v;
            if (u > v)
                swap(u, v);
            if (type == '+') {
                start[{u, v}] = t;
            } else {
                int L = start[{u, v}];
                seg.add(L, t - 1, {u, v});
                start.erase({u, v});
            }
        }
    }
    for (auto [edge, L] : start) {
        auto [u, v] = edge;
        seg.add(L, q, {u, v});
    }
    dfs(1, 1, q, seg, dsu, answer);
    for (int t = 1; t <= q; t++) {
        if (query[t])
            cout << answer[t] << '\n';
    }
}
```

5.10. Implicit Treap

```
// Mantiene las posiciones
struct Node {
    int val , pri , sz ;
    Node *l , *r;
    Node ( int v) {
        val = v;
        pri = rand () ;
        sz = 1;
        l = r = nullptr ;
    }
};
using pnode = Node *;
int sz ( pnode t) {
    return t ? t -> sz : 0;
}
void upd (pnode t) {
    if (t) t -> sz = 1 + sz (t -> l) + sz (t -> r);
}
void split ( pnode t , pnode &l , pnode &r , int k) {
    if (! t) {
        l = r = nullptr ;
        return ;
    }
    int leftSize = sz (t -> l);
    if ( leftSize >= k) {
        split (t -> l , l , t ->l , k) ;
        r = t;
    }
    else {
        split (t -> r , t ->r , r , k - leftSize - 1) ;
        l = t;
    }
    upd (t);
}
pnode merge ( pnode l , pnode r) {
    if (! l || !r ) return l ? l : r;

    if (l -> pri > r -> pri ) {
        l -> r = merge (l -> r , r);
        upd (l );
        return l;
    }
    else {
```

```cpp
        r -> l = merge (l , r ->l );
        upd (r );
        return r;
    }
}
void insert ( pnode & root , int pos , int val ) {
    pnode a , b;
    split ( root , a , b , pos );
    root = merge ( merge (a , new Node ( val )) , b) ;
}
void erase ( pnode & root , int pos ) {
    pnode a , b , c;
    split ( root , a , b , pos );
    split (b , b , c , 1) ;
    root = merge (a , c) ;
}
void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->val << " ";
    print(t->r);
}
int get(pnode t, int k) {
    if (!t) return -1;

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

    if (k == leftSize)
        return t->val;

    return get(t->r, k - leftSize - 1);
}
```

5.11. Ordered Set

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
     ordered_set;

void solve() {
    ordered_set st;

    st.insert(10);
    st.insert(30);
    st.insert(20);

    // 1. Elemento en la posicion K (0-indexed, de menor a mayor)
    // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
    auto it = st.find_by_order(1);
    if (it != st.end()) {
        int elemento = *it; // Devuelve 20 de forma segura
    }

    // 2. Cuantos elementos son estrictamente menores que X
    int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
}
// Sabrossus
```

5.12. Range Bit

```cpp
struct RangeBIT{
    int n;
    vector<ll> bit1, bit2;
    RangeBIT(int nn){
        n = nn;
        bit1.assign(n + 1,0);
        bit2.assign(n + 1,0);
    }
    void add_one(vector<ll> &b, int x, ll val){
        for (int i = x; i <= n; i+= (i&(-i))){
            b[i] += val;
```

```
        }
    }
    void add(int l, int r, ll val){
        add_one(bit1, l, val);
        add_one(bit1, r + 1, -val);
        add_one(bit2, l, l * val);
        add_one(bit2, r + 1, -(r + 1) * val);
    }
    ll sum_one(const vector<ll> &b, int x){
        ll ans = 0ll;
        for (int i = x; i > 0; i -= (i&(-i))){
            ans += b[i];
        }
        return ans;
    }

    ll sum(int x){
        return (x + 1) * sum_one(bit1,x) - sum_one(bit2, x);
    }
    ll query(int l, int r){
        return sum(r) - sum(l - 1);
    }
};
```

5.13. Sparse Table

```
const int N = 1e5;
const int LOG = 20;
int v[N]; //values
int st[N][LOG]; //Sparse Table
int lg[N]; // log values
int n;
void build(){
    for(int i = 0; i < n; i++) st[i][0] = v[i];

    for(int i = 1; i < LOG; i++){
        for(int j = 0; j < n - (1<<i) + 1; j++){
            st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
        }
    }
    lg[1] = 0;
    for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
}
int query(int L, int R){
    int k = lg[R-L+1];
    return min(st[L][k],st[R-(1<<k)+1][k]);
}
// v = [5, 2, 4, 7, 1, 3]
// query(1, 4)      [2, 4, 7, 1]
```

5.14. Treap

```
// Ordena los elementos por valor
struct Node {
    int key, pri, sz;
    Node *l, *r;
    Node(int k) {
        key = k;
        pri = rand();
        sz = 1;
        l = r = nullptr;
    }
};

using pnode = Node*;

int sz(pnode t) {
    return t ? t->sz : 0;
}

void upd(pnode t) {
    if (t) t->sz = 1 + sz(t->l) + sz(t->r);
}

void split(pnode t, pnode &l, pnode &r, int k) {
    if (!t) {
        l = r = nullptr;
        return;
    }

    int leftSize = sz(t->l);
```

```cpp

    if (leftSize >= k) {
        split(t->l, l, t->l, k);
        r = t;
    } else {
        split(t->r, t->r, r, k - leftSize - 1);
        l = t;
    }

    upd(t);
}

pnode merge(pnode l, pnode r) {
    if (!l || !r) return l ? l : r;

    if (l->pri > r->pri) {
        l->r = merge(l->r, r);
        upd(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        upd(r);
        return r;
    }
}

void insert(pnode &root, int pos, int val) {
    pnode a, b;
    split(root, a, b, pos);
    root = merge(merge(a, new Node(val)), b);
}

void erase(pnode &root, int pos) {
    pnode a, b, c;
    split(root, a, b, pos);
    split(b, b, c, 1);
    root = merge(a, c);
}

int get(pnode t, int k) {
    if (!t) return -1;

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

    if (k == leftSize)
        return t->key;

    return get(t->r, k - leftSize - 1);
}

void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->key << " ";
    print(t->r);
}
```

6. Dynamic Programing

6.1. Bounded Knapsack

```cpp
// bounded_knapsack para DPs del estilo :
// dp[j - cw] + cv , w-> pesov-> valor y quieres maximizar el valor
int n, we;
cin >> n >> we;
vector<int> v, m, w, dp;
v.resize(n);
m.resize(n);
w.resize(n);
dp.assign(we+1, 0);
int sum = 0;
for(int i  =0 ; i  < n ; i++){
    int a, b, c;
    cin >> a >> b >> c;
    v[i] = a;
    w[i] = b;
    m[i] = c;
```

```
    vector<int> old = dp;
    for(int r = 0; r <w[i] and r <= we ;r++){
        deque<pair<int,int>> dq;
        for(int j = r; j <= we ;j +=w[i]){
            int k = (j - r)/w[i];
            while(!dq.empty() and dq.front().first < k - m[i]){
                dq.pop_front();
            }
            int cur = old[j] - k * v[i];
            while(!dq.empty() && dq.back().second <= cur){
                dq.pop_back();
            }
            dq.push_back({k,cur});
            dp[j] = dq.front().second + k * v[i];
        }
    }
}
cout << dp[we];
```

6.2. Kadane 2D

```
int n;
cin >> n;
vector < vector < int >> v(n);
repl(i, 0, n) {
    repl(j, 0, n) {
        int x;
        cin >> x;
        v[i].push_back(x);
    }
}
int maxi = INT_MIN;
repl(ini, 0, n) {
    vector < int > pref(n, 0);
    repl(i, ini, n) {
        repl(j, 0, n) {
            pref[j] += v[i][j];
        }
        int cur = 0;
        repl(j, 0, n) {
            cur = max(pref[j], pref[j] + cur);
            maxi = max(maxi, cur);
        }
    }
}
```

6.3. Kadane 3D

```
int n;
cin >> n;

repl(i, 0, n) {
    repl(j, 0, n) {
        repl(k, 0, n) {
            cin >> v[i][j][k];
        }
    }
}
int maxi = INT_MIN;
repl(k_ini, 0, n) {
    vector < vector < int >> pref2(n, vector < int > (n, 0));
    repl(k, k_ini, n) {
        repl(i, 0, n) {
            repl(j, 0, n) {
                pref2[i][j] += v[i][j][k];
            }
        }
        repl(ini, 0, n) {
            vector < int > pref(n, 0);
            repl(i, ini, n) {
                repl(j, 0, n) {
                    pref[j] += pref2[i][j];
                }
                int cur = 0;
                repl(u, 0, n) {
                    cur = max(pref[u], cur + pref[u]);
                    maxi = max(maxi, cur);
                }
            }

        }
```

```
    }
}

cout << maxi << "\n";
```

6.4. SOS DP

```
void sos_dp(vector<ll>& dp, int K){
//  k es el tamanho del arreglo :v
    for(int bit = 0; bit < K; bit++){
        for(int mask = 0; mask < (1 << K); mask++){
            if(mask & (1 << bit)){
                dp[mask] += dp[mask ^ (1 << bit)];
//              si quieres que sea un sos dp de maximos
//              dp[mask] = max(dp[mask], dp[mask ^ (1 << bit)]);
            }
        }
    }
}
```

6.5. Fastfibo

```
#include <bits/stdc++.h>
#define ll long long

using namespace std;

const ll MOD = 1e9 + 7;

struct Matrix {
    ll a[2][2];
    Matrix() {
        memset(a, 0, sizeof(a));
    }
};

Matrix multiply(Matrix A, Matrix B) {
    Matrix C;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

Matrix power(Matrix A, ll n) {
    Matrix R;
    R.a[0][0] = 1;
    R.a[1][1] = 1;
    while (n) {
        if (n & 1)
            R = multiply(R, A);
        A = multiply(A, A);
        n >>= 1;
    }
    return R;
}

ll fibonacci(ll n) {
    if (n == 0) return 0;
    Matrix A;
    A.a[0][0] = 1;
    A.a[0][1] = 1;
    A.a[1][0] = 1;
    A.a[1][1] = 0;
    Matrix R = power(A, n - 1);
    return R.a[0][0];
}

int main() {
    ll n;
    cin >> n;
    cout << fibonacci(n) << '\n';
}
```

7. Flow

7.1. Dinic

```
struct Dinic
{
    const int INF = 1e9;

    struct flowEdge
    {
        int to, rev, f, cap;
    };

    vector<vector<flowEdge>> G;
    vector<int> work, lvl, Q;

    int S, T, nodes;

    Dinic(){}
    Dinic(int n, int s, int t){
        S = s;
        T = t;
        nodes = n;
        G.clear();
        G.resize(n);
        Q.resize(n);
    }
    void addEdge(int st, int en , int cap){
        flowEdge A = {en, (int)G[en].size(), 0, cap};
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].pb(A);
        G[en].pb(B);
    }
    int dfs(int v, int f){
        if(v == T or f == 0) return f;
        for(int &i = work[v]; i < G[v].size(); i++){
            flowEdge &e = G[v][i];
            int u = e.to;
            if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if(df){
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }
    bool bfs(){
        int qt = 0;
        Q[qt++] = S;
        lvl.assign(nodes,-1);
        lvl[S] = 0;
        for(int i = 0; i < qt; i++){
            int u = Q[i];
            for(flowEdge &e : G[u]){
                int v = e.to;
                if(e.cap <= e.f or lvl[v] != -1) continue;
                lvl[v] = lvl[u] + 1;
                Q[qt++] = v;
            }
        }
        return lvl[T] != -1;
    }
    int maxFlow(){
        int flow = 0;
        while(bfs()){
            work.assign(nodes,0);
            while(true){
                int df = dfs(S,INF);
                if(df == 0) break;
                flow += df;
            }
        }
        return flow;
    }

};
```

8. Graph Algorithms

8.1. Dijkstra

```
const int N = 1e5;

int vis[N];
vector<pair<int,int>> G[N];

void dijkstra(int x){
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,x});
    vis[x] = 0;
    while(!pq.empty()){
        auto y = pq.top();
        pq.pop();
        int u = y.second;
        int cost = y.first;
        for(auto v : G[u]){
            int cur = cost + v.second;
            if(cur < vis[v.first]){
                vis[v.first] = cur;
                pq.push({cur,v.first});
            }
        }
    }
}
```

8.2. Kruskal MST

```
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }
    }
};
int n;
vector<pair<int,pair<int,int>>> List;
int Kruskal_MST(){
    sort(List.begin(),List.end());
    DSU dsu(n);
    int mst = 0, t = 1;
    for(int i = 0; i < List.size(); i++){
        int w = List[i].first; //weigth
        int a = List[i].second.first; //at
        int b = List[i].second.second; //to
        if(dsu.find(a) != dsu.find(b)){
            dsu.join(a,b);
            mst+=w;
            t++;
        }
        if(t == n) break;
    }
```

```
    return mst;
}
```

8.3. Topological Sort Kahn Algorithm

```
const int N = 1e5;

int indegree[N];
vector<int> G[N];
int n;
vector<int> Kahn(){
    queue<int> q;
    vector<int> ans;
    for(int i = 0; i < n; i++){
        if(indegree[i] == 0) q.push(i);
    }
    while (!q.empty()){
        int v = q.front();
        q.pop();
        ans.push_back(v);
        for(int u : G[v]){
            indegree[u]--;
            if(indegree[u] == 0){
                q.push(u);
            }
        }
    }
    //if(ans.size() != n) grafo no DAG
    return ans;
}
```

8.4. Kosaraju

```
struct Kosaraju {
    int n, components;
    vector<vector<int>> graph, invgraph;
    vector<int> vis, order, component;

    Kosaraju(int n) : n(n) {
        graph.resize(n + 1);
        invgraph.resize(n + 1);
        vis.resize(n + 1);
        component.resize(n + 1);
    }

    void add_edge(int u, int v) {
        graph[u].push_back(v);
        invgraph[v].push_back(u);
    }

    void dfs1(int u) {
        vis[u] = 1;
        for (int v : graph[u]) {
            if (!vis[v])
                dfs1(v);
        }
        order.push_back(u);
    }

    void dfs2(int u, int id) {
        vis[u] = 1;
        component[u] = id;
        for (int v : invgraph[u]) {
            if (!vis[v])
                dfs2(v, id);
        }
    }
    void build() {
        for (int u = 1; u <= n; u++) {
            if (!vis[u])
                dfs1(u);
        }
        reverse(order.begin(), order.end());
        fill(vis.begin(), vis.end(), 0);
        components = 0;
        for (int u : order) {
            if (!vis[u]) {
                components++;
                dfs2(u, components);
            }
        }
```

```
    }
};
```

8.5. 2SAT

```
struct TwoSAT {
    int n, nodes;
    vector<vector<int>> graph, invgraph;
    vector<int> vis, order, component;
    vector<bool> ans;

    TwoSAT(int n) : n(n), nodes(2 * n) {
        graph.resize(nodes);
        invgraph.resize(nodes);
        vis.resize(nodes);
        component.resize(nodes);
        ans.resize(n);
    }

    int id(int x, bool value) {
        return 2 * x + value;
    }

    int neg(int x) {
        return x ^ 1;
    }

    void add_implication(int a, int b) {
        graph[a].push_back(b);
        invgraph[b].push_back(a);
    }

    void add_or(int a, int b) {
        add_implication(neg(a), b);
        add_implication(neg(b), a);
    }

    void add_xor(int a, int b) {
        add_or(a, b);
        add_or(neg(a), neg(b));
    }

    void add_equivalence(int a, int b) {
        add_implication(a, b);
        add_implication(b, a);
        add_implication(neg(a), neg(b));
        add_implication(neg(b), neg(a));
    }

    void dfs1(int u) {
        vis[u] = 1;
        for (int v : graph[u]) {
            if (!vis[v])
                dfs1(v);
        }
        order.push_back(u);
    }

    void dfs2(int u, int id) {
        vis[u] = 1;
        component[u] = id;
        for (int v : invgraph[u]) {
            if (!vis[v])
                dfs2(v, id);
        }
    }

    bool solve() {
        for (int u = 0; u < nodes; u++) {
            if (!vis[u])
                dfs1(u);
        }
        reverse(order.begin(), order.end());
        fill(vis.begin(), vis.end(), 0);
        int cnt = 0;
        for (int u : order) {
            if (!vis[u]) {
                dfs2(u, cnt++);
            }
        }
        for (int i = 0; i < n; i++) {
            if (component[2 * i] == component[2 * i + 1])
                return false;
            ans[i] = component[2 * i] > component[2 * i + 1];
```

```
        }
        return true;
    }
};
```

9. Tree Algorithms

9.1. Euler Tour

```
const int N = 1e5 + 10;
vector<int> path;
vector<int> G[N];
void EulerTour(int node, int parent){
    path.pb(node);
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u,node);
        }
    }
    path.pb(node);
}

const int N = 1e5 + 10;
int tin[N];
int tout[N];
int flat[N];
int timer = 1;
void EulerTour(int node, int parent){
    tin[node] = timer;
    flat[timer] = node;
    timer++;
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u, node);
        }
    }
    tout[node] = timer;
}
```

9.2. Binary Lifting

```
const int N = 1e5;
const int LOG = 20;
int up[N][LOG];
vector<int> G[N];
//Find parents
void dfs(int u){
    for(int v : G[u]){
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_K_parent(int a, int k){
    for(int j = LOG-1; j >= 0; j++){
        if(k&(1<<j)){
            a = up[a][j];
        }
    }
    return a;
}
```

9.3. Centroid Decomposition

```
//EJERCICIO IMPLEMENTACION XENIA AND TREE
//Nodo rojo mas cercano a un nodo v

const int N = 2e5+10;
vector<int>G[N];
int tag[N]; // time of discovery of centrodi
int fat[N]; // father in centroid decomposition
int szt[N]; // size of subtree

```

```
int best[N];
int niv[N];
int dist[30][N]; //dist[nivel][nodo];

void dfs_level(int node, int p, int niv, int prof){
    dist[niv][node] = prof;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            dfs_level(y, node, niv, prof+1);
        }
    }
}
// -----------------CENTROID-------------------------
int calcsz(int node, int p){
    szt[node] = 1;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            szt[node] += calcsz(y, node);
        }
    }
    return szt[node];
}
int ccnt = 0;
void cdfs(int node, int p, int nivel = 0, int sz = -1){
    if(sz < 0) sz = calcsz(node, -1);
    for(auto y : G[node]){
        if(tag[y] < 0 and szt[y] * 2 >= sz){
            szt[node] = 0;
            cdfs(y, p, nivel, sz);
            return;
        }
    }
    tag[node] = ccnt++;
    fat[node] = p;
    niv[node] = nivel;
    dfs_level(node, -1, nivel, 0);
    for(auto y : G[node]){
        if(tag[y] < 0){
            cdfs(y, node, nivel+1);
        }
    }
}

int n;
void centroid(){
    ccnt = 0;
    repl(i,1,n+1){
        tag[i] = -1;
    }
    cdfs(1, -1);
}

//---------------------------------------------

void update(int node, int rojo){
    int nivel = niv[node];
    int cost = dist[nivel][rojo];
    best[node] = min(best[node], cost);
    int p = fat[node];
    if(p > 0){
        update(p, rojo);
    }
}

int improve(int node, int cur){
    if(node <= 0){
        return 1e9;
    }
    int nivel = niv[node];
    int cost = best[node] + dist[nivel][cur];
    int p = fat[node];
    return min(cost, improve(p, cur));
}

void solve(){
    int qq;
    cin >> n >> qq;
    repl(i,0,30){
        repl(j,1,n+1){
            dist[i][j] = -1;
            best[j] = 1e9;
        }
    }
    repl(i,0,n-1){
        int a, b;
        cin >> a >> b;
```

```
            G[a].push_back(b);
            G[b].push_back(a);
        }
        centroid();
        update(1, 1);
        while(qq--){
            int q, node;
            cin >> q >> node;
            if(q == 1){
                update(node, node);
            }
            else{
                int res = best[node];
                int res2 = improve(node, node);
                cout << min(res, res2) << '\n';
            }
        }
    }

    int main(){
        velocito;
        int t = 1;
        // cin >> t;
        while(t--){
            solve();
        }
        return 0;
    }
    //Sabrossus
    //TC Argentina
```

9.4. HLD

```
struct HLD {
        int n, act;
        vector<vector<int>> adj;
        vector<int> parent, depth, heavy, head, pos;
        vector<int> val;
        vector<int> seg;
        int tam;
        HLD(int n) : n(n) {
                adj.resize(n+1);
                parent.resize(n+1);
                depth.resize(n+1);
                heavy.assign(n+1, -1);
                head.resize(n+1);
                pos.resize(n+1);
                val.resize(n+1);
                act = 0;
        }

        void add_edge(int u, int v) {
                adj[u].push_back(v);
                adj[v].push_back(u);
        }

        int dfs(int u, int p) {
                parent[u]=p;
                int size=1, max_sub=0;
                for (int v : adj[u]){
                        if(v==p)continue;
                        depth[v]=depth[u]+1;
                        int sub=dfs(v,u);
                        if(sub>max_sub){
                                max_sub=sub;
                                heavy[u]=v;
                        }
                        size+=sub;
                }
                return size;
        }

        void decompose(int u, int h) {
                head[u]=h;
                pos[u]=act++;
                if(heavy[u]!=-1)decompose(heavy[u], h);
                for(int v: adj[u]){
                        if(v!=parent[u] and v!=heavy[u])decompose(v,v);
                }
        }

        void build(){
                tam=1;
                while (tam < n) tam<<=1;
```

```cpp
			seg.assign(2*tam,inf);
			for (int i = 1; i <= n; i++){
				seg[tam+pos[i]]=val[i];
			}
			for (int i = tam - 1; i > 0; i--){
				seg[i]=max(seg[2*i],seg[2*i+1]);
			}
		}

		void update(int u, int value) {
			int i=tam+pos[u];
			seg[i]=value;
			for(i>>=1; i; i>>=1){
				seg[i]=max(seg[2*i],seg[2*i+1]);
			}
		}

		int query(int l, int r) {
			int res=inf;
			l+=tam;
			r+=tam;
			while(l <= r){
				if(l&1)res=max(res,seg[l++]);
				if(!(r&1))res=max(res,seg[r--]);
				l>>=1;
				r>>=1;
			}
			return res;
		}

		void init(){
			int root=1;
			dfs(root, 0);
			decompose(root,root);
			build();
		}

		int query_path(int a, int b){
			int res=inf;
			while(head[a]!=head[b]){
				if(depth[head[a]]<depth[head[b]])swap(a,b);
				res=max(res,query(pos[head[a]],pos[a]));
				a=parent[head[a]];
			}
			if(depth[a]>depth[b])swap(a,b);
			res=max(res,query(pos[a],pos[b]));
			return res;
		}
};

void solve(){
		int n,q;
		cin>>n>>q;
		HLD hld(n);
		for(int i = 1 ; i <= n ; i++){
			cin>>hld.val[i];
		}
		for(int i = 0; i  < n-1 ; i++){
			int a,b;
			cin>>a>>b;
			hld.add_edge(a,b);
		}
		hld.init();
		while(q--){
			int t;
			cin>>t;
			if(t==1){
				int s,x;
				cin>>s>>x;
				hld.update(s,x);
			}
			else{
				int a,b;
				cin>>a>>b;
				cout<<hld.query_path(a,b)<<' ';
			}
		}
}
```

9.5. Lowest Common Ancestor

```cpp
const int N = 1e5;
const int LOG = 20;
vector<int> G[N];
```

```
int depth[N];
int up[N][LOG];
// Assign levels and get parents
void dfs(int u){
    for(int v : G[u]){
        depth[v] = depth[u] + 1;
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_lca(int a, int b){
    if(depth[a] < depth[b]) swap(a,b);
    int k = depth[a] - depth[b];
    for(int j = LOG-1; j >= 0; j--){
        if(k & (1<<j)){
            a = up[a][j];
        }
    }
    if(a == b) return a;
    for(int j = LOG-1; j >= 0; j--){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

10. Utilities

10.1. Prime Numbers

```
   2    3    5    7   11   13   17   19   23   29
  31   37   41   43   47   53   59   61   67   71
  73   79   83   89   97  101  103  107  109  113
 127  131  137  139  149  151  157  163  167  173
 179  181  191  193  197  199  211  223  227  229
 233  239  241  251  257  263  269  271  277  281
 283  293  307  311  313  317  331  337  347  349
 353  359  367  373  379  383  389  397  401  409
 419  421  431  433  439  443  449  457  461  463
 467  479  487  491  499  503  509  521  523  541
 547  557  563  569  571  577  587  593  599  601
 607  613  617  619  631  641  643  647  653  659
 661  673  677  683  691  701  709  719  727  733
 739  743  751  757  761  769  773  787  797  809
 811  821  823  827  829  839  853  857  859  863
 877  881  883  887  907  911  919  929  937  941
 947  953  967  971  977  983  991  997 1009 1013
1019 1021 1031 1033 1039 1049 1051 1061 1063 1069
1087 1091 1093 1097 1103 1109 1117 1123 1129 1151
1153 1163 1171 1181 1187 1193 1201 1213 1217 1223
1229 1231 1237 1249 1259 1277 1279 1283 1289 1291
1297 1301 1303 1307 1319 1321 1327 1361 1367 1373
1381 1399 1409 1423 1427 1429 1433 1439 1447 1451
1453 1459 1471 1481 1483 1487 1489 1493 1499 1511
1523 1531 1543 1549 1553 1559 1567 1571 1579 1583
1597 1601 1607 1609 1613 1619 1621 1627 1637 1657
1663 1667 1669 1693 1697 1699 1709 1721 1723 1733
1741 1747 1753 1759 1777 1783 1787 1789 1801 1811
1823 1831 1847 1861 1867 1871 1873 1877 1879 1889
1901 1907 1913 1931 1933 1949 1951 1973 1979 1987

      970'997␣␣␣␣␣␣␣971'483    921'281'269    999'279'733
1'000'000'009␣1'000'000'021 1'000'000'409␣1'005'012'527
1000551285057160091l
```

10.2. ASCII TABLE

```
ASCII QUICK REFERENCE
============================================================

CHARACTER RANGES
------------------------------------------------------------
'0' ... '9'    48 ... 57     -> c - '0'
```

```
'A' ... 'Z'     65 ... 90      -> c - 'A'
'a' ... 'z'     97 ... 122     -> c - 'a'

RELATIONSHIPS
-----------------------------------------------------------
'a' - 'A' = 32
'A' + 32    = 'a'
'a' - 32    = 'A'

DIGITS
-----------------------------------------------------------
'0' = 48        '5' = 53
'1' = 49        '6' = 54
'2' = 50        '7' = 55
'3' = 51        '8' = 56
'4' = 52        '9' = 57

UPPERCASE
-----------------------------------------------------------
A 65    B 66    C 67    D 68    E 69    F 70    G 71
H 72    I 73    J 74    K 75    L 76    M 77    N 78
O 79    P 80    Q 81    R 82    S 83    T 84    U 85
V 86    W 87    X 88    Y 89    Z 90

LOWERCASE
-----------------------------------------------------------
a 97    b 98    c 99    d 100   e 101   f 102   g 103
h 104   i 105   j 106   k 107   l 108   m 109   n 110
o 111   p 112   q 113   r 114   s 115   t 116   u 117
v 118   w 119   x 120   y 121   z 122

COMMON SYMBOLS
-----------------------------------------------------------
SPACE   32      ! 33      "␣34␣␣␣␣␣␣#␣35␣␣␣␣␣␣$␣36
%␣37␣␣␣␣␣␣␣␣␣␣␣␣&␣38␣␣␣␣␣␣'␣39␣␣␣␣␣␣(␣40␣␣␣␣␣␣)␣41
*␣42␣␣␣␣␣␣␣␣␣␣␣␣+␣43␣␣␣␣␣␣,␣44␣␣␣␣␣␣-␣45␣␣␣␣␣␣.␣46
/␣47␣␣␣␣␣␣␣␣␣␣␣␣:␣58␣␣␣␣␣␣;␣59␣␣␣␣␣␣<␣60␣␣␣␣␣␣=␣61
>␣62␣␣␣␣␣␣␣␣␣␣␣␣?␣63␣␣␣␣␣␣@␣64␣␣␣␣␣␣[␣91␣␣␣␣␣␣\␣92
]␣93␣␣␣␣␣␣␣␣␣␣␣␣^␣94␣␣␣␣␣␣_␣95␣␣␣␣␣␣`␣96␣␣␣␣␣␣{␣123
|␣124␣␣␣␣␣␣␣␣␣␣␣}␣125␣␣␣␣␣~␣126

USEFUL␣TRICKS
-----------------------------------------------------------
int␣x␣=␣c␣-␣'0';␣␣␣␣␣␣␣␣//␣digit␣->␣integer
char␣c␣=␣'0'␣+␣x;␣␣␣␣␣␣␣//␣integer␣->␣digit

int␣x␣=␣c␣-␣'a';␣␣␣␣␣␣␣␣//␣lowercase␣->␣[0,25]
int␣x␣=␣c␣-␣'A';␣␣␣␣␣␣␣␣//␣uppercase␣->␣[0,25]

islower(c)
isupper(c)
isdigit(c)
isalpha(c)
isalnum(c)
isspace(c)
tolower(c)
toupper(c)
===========================================================
```

10.3. Int128

```
// --- LECTURA B SICA ---
__int128 read128() {
    string s;
    cin >> s;
    __int128 n = 0;
    int sign = 1;
    size_t start = 0;

    if (!s.empty() && s[0] == '-') {
        sign = -1;
        start = 1;
    }

    for (size_t i = start; i < s.length(); ++i) {
        n = n * 10 + (s[i] - '0');
    }

    return n * sign;
}

// --- IMPRESI N B SICA ---
void print128(__int128 n) {
    if (n == 0) {
```

```
        cout << 0;
        return;
    }
    if (n < 0) {
        cout << '-';
        n = -n;
    }
    string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}
```

10.4. Merge Sort

```
// Rango de uso: 0-indexed.
// Llamar en el solve como: mergeSort(v, 0, n - 1);
ll ans = 0;

void mergeSort(vector<ll> &v, int low, int high) {
    if (low == high) return;

    int mid = (low + high) >> 1;
    mergeSort(v, low, mid);
    mergeSort(v, mid + 1, high);

    queue<ll> L, H;
    for (int i = low; i < mid + 1; i++) {
        L.push(v[i]);
    }
    for (int i = mid + 1; i < high + 1; i++) {
        H.push(v[i]);
    }

    for (int i = low; i < high + 1; i++) {
        if (L.size() == 0) {
            v[i] = H.front();
            H.pop();
        }
        else if (H.size() == 0) {
            v[i] = L.front();
            L.pop();
        }
        else {
            if (L.front() <= H.front()) {
                v[i] = L.front();
                L.pop();
            }
            else {
                v[i] = H.front();
                H.pop();
                ans += L.size(); // Inversion detectada
            }
        }
    }
}
```

10.5. Preffix Sum 2D

```
//1 indexed
const int N = 1005;
int v[N][N];
int acu[N][N];
int n, m;

void solve() {
    //limpiar la acu para cada TC

    // Construccion: acumula 1 si el elemento es exactamente 1
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int val = (v[i][j] == 1 ? 1 : 0);
            acu[i][j] = val + acu[i - 1][j] + acu[i][j - 1] - acu[i - 1][j - 1];
        }
    }

    // Esquinas de consulta:
    // (a, b) -> Superior Izquierda
```

```
    // (c, d) -> Inferior Derecha
    int a = 2, b = 2, c = 4, d = 4; // Valores de ejemplo

    int ans = acu[c][d] - acu[a - 1][d] - acu[c][b - 1] + acu[a - 1][b - 1];
    cout << ans << '\n';
}
```

10.6. Priority Queue Min Heap

```
priority_queue<int, vector<int>, greater<int>> pq;
```

10.7. Random

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// mt19937_64 para 64 bits usar uint64_t o unsigned long long
cout << rng() << endl;
shuffle(v.begin(),v.end(),rng);
```

10.8. Plantilla Python

```
import sys
from bisect import bisect_left, bisect_right, insort
from collections import defaultdict, Counter, deque
from heapq import heappush, heappop, heapify
from math import gcd, lcm, isqrt

input = sys.stdin.readline


def solve():
    n = int(input()) #leer numero
    a,b,c= map(int, input().split()) #leer numeros
    s = input().strip() #leer cadenas
    v = list(map(int, input().split())) #leer lista de enteros


t = 1
# t = int(input())
for _ in range(t):
    solve()

# C++                             Python

# vector                          list
# array                           list
# string                          str

# set                             set
# map                             dict
# multiset                        Counter
# unordered_map                   dict
# unordered_set                   set

# priority_queue                  heapq
# queue                           deque
# deque                           deque

# lower_bound                     bisect_left
# upper_bound                     bisect_right
# sort                            list.sort()
# binary_search                   bisect_left/right

# pair                            tuple
# struct                          class / dataclass

# gcd                             math.gcd
# lcm                             math.lcm

# push_back                       append
# push_front                      appendleft
# pop_back                        pop
# pop_front                       popleft

# size()                          len()
# empty()                         not x
```

10.9. Pragma

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```